



ENGINEERING HANDOVER

Final Project Leaderboard

Technical Specification & Build Guide

A public-vote awards platform for the Claude Course for Professionals cohort — one standard project card, four award categories, participant-submitted demo videos with admin review, and a live results reveal.

PREPARED BY

Meska AI
Learning & Delivery

TARGET DOMAIN

leaderboard.meska.ai

STATUS

Prototype complete

CLASSIFICATION

Internal — Confidential

SECTION 01

What This Is

The product, the deliverables, and how to treat them.

A public-vote leaderboard for the cohort's final projects. Participants submit a project with a demo video; the cohort and the public vote across four award categories; an admin reveals the winners at a closing ceremony.

The core design principle — set in the original plan and confirmed by the CEO: every project is presented with **one identical set of fields**. There are no per-category criteria. Voters read the same card for each project and decide for themselves which project deserves each award. This keeps the vote fair: no project can be dressed up differently to chase a category.

What you are receiving

FILE	WHAT IT IS
Option B (Backend Demo) .html	The reference implementation. Full UX for the account-based version, running on a mock in-browser store. This is the spec — build this against a real backend.
Option A (Links Only) .html	Simpler fallback: no accounts, admin enters everything, demo videos are pasted links. Useful as a contingency if the backend slips.
meska-leaderboard-schema.sql	A first-pass Postgres/Supabase schema with row-level security. Reviewed but never run against a live project.
This document	Spec, data model, and build checklist.

Both HTML files are **single self-contained files** — no build step, no dependencies, logo embedded as base64. Open directly in a browser. All state lives in `localStorage`.

Important: these prototypes are throwaway. Do not try to grow Option B into production by bolting a backend onto its [Store](#) module. Use it as the functional and visual specification, and build properly.

Decisions Already Made

Settled in the CEO review, 29 July 2026 — do not re-litigate without Learning & Delivery.

#	DECISION	WHY
1	Participants upload their own projects — not the admin	Each person owns their submission, so nobody can later claim "I didn't write that." Submissions pass through admin filtration before publishing.
2	Per-project vote counts are hidden from voters	Voters see the total votes cast and a per-award total, never how an individual project is doing. Prevents bandwagon voting.
3	The leaderboard page is locked until an admin reveals it	The reveal is the ceremony moment.
4	No winners shown anywhere before that reveal	Same reason.
5	Demo videos appear as real thumbnails and play in-page	People vote with their eyes first; a bare link loses them, and a new tab loses the page.
6	Cohort Choice is restricted to an admin-managed list	Only the cohort should decide its own choice award.
7	That same list controls who may submit	One list, two powers — simpler to administer.
8	Re-submission resets: back to pending, votes cleared	A replaced project must not inherit votes earned by the previous version.
9	Registration captures leads	Name, email, mobile, job title, company, LinkedIn.

Open decisions — need an answer before launch

1 · Can one voter give the same project more than one award?

Currently yes. Confirm this is intended.

2 · Can participants vote for their own project?

There is a `selfVotingAllowed` setting but **it is not enforced anywhere** — it is a reminder flag only. Decide the rule and either enforce it or remove the setting.

3 · Project ordering leaks ranking.

The homepage sorts by vote count, so the leading project sits top-left even though numbers are hidden. This contradicts the original plan's anti-bias randomisation. Either accept it, or switch to a per-voter random order — a one-line change, see Section 05.

SECTION 03

Roles & Permissions

Three tiers of access, driven by one email list plus an admin flag.

	PUBLIC (REGISTERED)	COHORT MEMBER	ADMIN
Browse projects, watch demos	✓	✓	✓
Vote Best ROI / Technical / Creative	✓	✓	✓
Vote Cohort Choice	—	✓	if listed
Submit a project	—	✓	if listed
See the leaderboard before reveal	—	—	✓ preview
Approve / reject submissions	—	—	✓
Manage cohort list, settings, reports	—	—	✓

Cohort membership = an email on the admin-managed list. One list, two powers: submit rights and Cohort Choice voting rights. Anyone can register and vote the other three awards.

Admin is a flag on the user account — `isAdmin` in the prototype, `profiles.is_admin` in the schema. There is no self-service path to becoming an admin; set it manually in the database. **Bootstrap:** register normally, then flip the flag.

Pages

ROUTE	WHO SEES IT	PURPOSE
Vote (home)	Everyone	Project grid, vote totals box, voting
Leaderboard	Everyone (locked until reveal)	Rankings, winners celebration
About	Everyone	How the awards work, the nine card fields, voting rules
Submit	Cohort members only	Instructions + submission form
Admin	Admins only	Review queue, leads, reports, settings, cohort list

The prototype uses hash routing (`#vote` , `#board` , `#about` , `#submit` , `#admin`) with all views in one document. In production these should be real routes.

Page Behaviour

What each screen must do.

Vote page (home)

- **Vote totals box**, always visible: overall total, plus a count per award. **Never per project.**
- **Project grid**: cards with a 16:9 video thumbnail, project name, tagline, participant. Clicking opens the detail view.
- **Project detail**: all nine fields, large video thumbnail, and the four vote buttons.
- Vote buttons read `Vote` or `Your vote` — **no counts**.
- A "viewed everything" gate: by default voters must open every project before their first vote is accepted (`requireViewAll`).

Leaderboard

Locked by default — a padlock and an explanation, with **no vote data at all**. Once `resultsRevealed` is true:

1. Countdown overlay: `3 → 2 → 1 → 🎆 Congratulations!` (900 ms per beat)
2. Fireworks canvas animation starts on the congratulations beat
3. Winners appear: four cards side by side, each with participant **photo**, trophy, **project name**, **participant name**, and vote count
4. Full ranked columns below, with per-project counts
5. A "Replay the reveal" button re-runs the sequence

Admins see the board **before** reveal with an "Admin preview" banner, so numbers can be sanity-checked privately. Fireworks are skipped entirely under `prefers-reduced-motion`, and stop when the user leaves the tab.

Submit page

Two halves: a "what we need from you" brief (six numbered points covering the video, photo, problem/solution, ROI, tech stack, creative twist), then the form. Fields map 1:1 to the project record, and include a **video file picker** with progress, a **fallback link field**, and a **photo picker**.

If the participant already has a submission, the form pre-fills it and shows current status. Re-submitting an approved project triggers a confirmation warning that votes will be cleared.

Admin

1. **Submissions awaiting review** — video thumbnail, details, Approve / Reject (with reason) / Edit
2. **Registered voters** — the lead list, CSV export
3. **Report: votes per day** — date, total, and a column per award, CSV export
4. **Report: who voted for what** — name, email, mobile, and the project picked in each award, CSV export
5. **Voting settings** — voting open, require-view-all, self-voting flag, reveal results publicly, rules note
6. **Cohort members** — the email list
7. **Projects & approvals** — full CRUD, status toggle

SECTION 05

The Standard Project Card

Nine fields, identical for every project. This is the heart of the product.

Do not add per-category fields. The whole fairness argument rests on every project being described the same way.

#	FIELD	KEY	NOTES
1	Project name + tagline	<code>name</code> , <code>tagline</code>	One line, plain language
2	Participant	<code>participant</code> , <code>photo</code>	One individual, not a team. Photo shown if they win.
3	Problem statement	<code>problem</code>	2–3 sentences
4	Solution overview	<code>solution</code>	2–3 sentences
5	Demo	<code>demoLink</code> , <code>videoFileName</code>	30–60s video. Anchors the whole thing.
6	Impact / ROI note	<code>impact</code>	A number, even an estimate. Anchors Best ROI .
7	Tech stack & hard problem	<code>techStack[]</code> , <code>hardProblem</code>	Anchors Best Technical
8	The creative twist	<code>creativeTwist</code>	Anchors Best Creative
9	Live link / repo	<code>liveLink</code>	Optional

Plus system fields: `id` , `status` , `submittedBy` , `rejectReason` , `sample` .

Award categories

Fixed, four, hard-coded:

```
[
  { id: 'roi',      label: 'Best ROI' },
  { id: 'technical', label: 'Best Technical' },
  { id: 'creative',  label: 'Best Creative' },
  { id: 'cohort',    label: 'Cohort Choice' }, // restricted
]
```

Ordering

```
// Highest-voted first, ties broken by a per-voter shuffle.
// To restore the original anti-bias behaviour, drop the sort
// and return the shuffle alone.
function orderProjects(projects) {
  const shuffled = Store.seededShuffle(projects, Store.getVoterId());
  const totalFor = p => CATEGORIES.reduce((n, c) => n + (tally(c.id)[p.id] || 0), 0);
  return shuffled
    .map((p, i) => ({ p, i, n: totalFor(p) }))
    .sort((a, b) => (b.n - a.n) || (a.i - b.i))
    .map(x => x.p);
}
```

Data Model — Records

Shapes taken from the working prototype.

Project

```
{
  id: string
  name: string
  tagline: string
  participant: string           // display name of the submitter
  photo: string                // data URL in prototype → storage URL in production
  problem: string
  solution: string
  demoType: 'video' | 'screenshots'
  demoLink: string             // YouTube / Drive / Loom / Vimeo
  videoFileName: string        // original filename of an uploaded video
  demoShots: string[]          // optional screenshot labels
  impact: string
  techStack: string[]
  hardProblem: string
  creativeTwist: string
  liveLink: string
  status: 'pending' | 'approved' | 'rejected'
  rejectReason: string
  submittedBy: string          // owner email – one live project per person
  sample: boolean              // demo seed data only; drop in production
}
```

User / lead

```
{
  email: string                // primary key, lowercased
  name: string
  title: string
  company: string
  mobile: string
  linkedin: string
  isAdmin: boolean
  createdAt: ISO8601
  // password is plaintext in the prototype ONLY – see Section 08
}
```

Data Model — Votes & Settings

How a vote is recorded, and the switches that control the event.

Vote

Prototype shape — one vote per person per category, keyed by identity:

```
votes    = { [categoryId]: { [voterEmail]: projectId } }
voteMeta = { [categoryId]: { [voterEmail]: ISO8601 } } // cast timestamp
```

In production this should simply be a `votes` table with a `UNIQUE (category, voter_id)` constraint and a `created_at`. That uniqueness constraint **is** the "one vote per person per category" rule — enforce it in the database, not the UI.

Cohort list — a list of lowercased emails. Grants submit rights **and** Cohort Choice voting rights.

Settings (single row)

```
{
  votingOpen: boolean      // master switch
  requireViewAll: boolean  // must open every project before voting
  selfVotingAllowed: boolean // NOT ENFORCED — see open decisions
  resultsRevealed: boolean // unlocks the leaderboard for everyone
  rulesNote: string        // free text shown on the vote page
  eventName: string
}
```

What Must Change For Production

The prototype is a specification, not a foundation.

Authentication — highest priority. The prototype stores **passwords in plain text** in `localStorage`. This is deliberate and clearly commented, because it is a throwaway demo with no server. It must never ship.

Use a real auth provider (Supabase Auth, Auth0, Clerk — the schema assumes Supabase). Requirements:

- Email + password registration capturing the lead fields
- Session persistence
- `is_admin` flag on the profile, settable only in the database
- Decide on email confirmation. For a live event, confirmation-on is safer for lead quality but slower on the day; confirmation-off gets people voting instantly. Ops preference so far has been **off** for speed.

File storage

Two uploads, neither of which really exists yet:

- **Demo video** — up to ~200 MB. In the prototype the file never leaves the browser; it is held as an in-memory object URL that dies on reload. Needs real object storage with resumable upload, a size/type limit, and ideally transcoding or at least a poster frame.
- **Participant photo** — the prototype downscales to a 220×220 JPEG data URL (~3 KB) via canvas, which is a reasonable trick to keep, but it should still end up in object storage.

Also handle: what happens to the old video when someone re-submits.

Realtime

The leaderboard should update live during the ceremony without refreshes. The prototype fakes this with a 4-second poll plus same-browser `storage` events. Use proper realtime subscriptions on the votes and projects tables.

Server-side rule enforcement

Everything below is currently enforced in the browser only and **must** move server-side:

RULE	ENFORCEMENT
One vote per person per category	<code>UNIQUE (category, voter_id)</code>
Cohort Choice restricted	Trigger checking the voter's email against the cohort list — see the schema's <code>check_cohort_eligibility()</code>
Only cohort members may submit	Row-level security on insert
Only admins approve / reject / configure	RLS via an <code>is_admin()</code> helper
Voting closed	Reject writes when <code>voting_open</code> is false
Results hidden before reveal	Do not send vote counts to non-admin clients at all while <code>results_revealed</code> is false

That last one matters. In the prototype the counts are present in the client and merely not rendered — anyone with devtools can read them. In production, the API must not return them.

Re-submission behaviour

Implement as a transaction: replace the project record, set status back to `pending`, clear `reject_reason`, and **delete every vote pointing at that project**. Reference implementation is `Store.submitProject()`.

Build Checklist

Acceptance criteria, grouped by area.

Foundations

- ☐ Auth with lead capture (name, email, mobile, title, company, LinkedIn)
- ☐ Profiles table with `is_admin`
- ☐ Cohort email list, admin-managed
- ☐ Settings singleton
- ☐ Object storage for video + photo

Participant

- ☐ Submit page: instructions + form, gated to cohort members
- ☐ Video upload with real progress; link fallback
- ☐ Photo upload
- ☐ Status view: pending / live / sent back with feedback
- ☐ Re-submission clears votes and returns to pending

Ceremony

- ☐ Leaderboard locked until reveal
- ☐ 3-2-1 countdown → congratulations → winners
- ☐ Winners: photo + project name + participant, side by side
- ☐ Fireworks background, reduced-motion safe
- ☐ Replay control
- ☐ Verify on the actual projector and resolution

Deployment

- **Domain:** `leaderboard.meska.ai`
- Static front end (Netlify / Vercel / Cloudflare Pages) + managed backend is sufficient; there is no heavy server-side compute.
- Add the production origin to the auth provider's allowed redirect URLs.
- **Load is trivial** — one cohort, tens of concurrent voters. Do not over-engineer. The only spike is everyone opening the leaderboard at reveal time.
- Keep a fallback: if the backend fails on the day, Option A runs from a single file with no network at all.

Voter

- ☐ Project grid with real video thumbnails
- ☐ Project detail with all nine fields
- ☐ In-page video player (YouTube / Vimeo / Drive embeds + uploaded files)
- ☐ Four vote buttons, no counts, toggleable
- ☐ Cohort Choice blocked for non-members, with a clear message
- ☐ Vote totals box: overall + per award only
- ☐ View-all gate

Admin

- ☐ Review queue with Approve / Reject + reason / Edit
- ☐ Leads table + CSV
- ☐ Votes-per-day report + CSV
- ☐ Who-voted-for-what report + CSV
- ☐ Settings incl. reveal toggle
- ☐ Private board preview before reveal

SECTION 10

Prototype Architecture & Limitations

How to read the reference file, and what not to trust in it.

File structure

```
<style>          design tokens + all component CSS
<body>
  .demo-banner   (Option B only)
  header.topnav  logo, view tabs, auth area
  section.view#view-vote
  section.view#view-board      + fireworks canvas + countdown overlay
  section.view#view-about
  section.view#view-submit     (Option B only)
  section.view#view-admin
  modals: auth, video player, project editor
<script>
  Store          all state + persistence + rules
  render helpers renderVideoThumb / renderTile / renderCard / renderVotePills
  VideoPlayer    in-page embed player
  view switcher  setView / onViewChange
  Auth           register / login / gating          (Option B only)
  Submit module  photo + video pickers              (Option B only)
  Vote module    grid, detail, voting
  Leaderboard    lock, winners, fireworks, reveal
  Admin          queue, leads, reports, settings, CRUD
```

Everything reads and writes through `Store`, and every module re-renders via `Store.subscribe(fn)`. That seam is why the prototype could be repointed at a real API — but again, treat it as spec, not a starting codebase. `localStorage` keys are namespaced per file (`meskaA_lb_*`, `meskaB_demo_*`) so the two prototypes never collide.

Known limitations of the prototypes

#	LIMITATION
1	No backend. Everything is <code>localStorage</code> ; nothing syncs across devices.
2	Plaintext passwords in Option B's mock store. Demo only.
3	Uploads are simulated. The video never leaves the browser and does not survive a reload.
4	Vote counts are present client-side even when hidden — fix in the API layer.
5	<code>selfVotingAllowed</code> does nothing.
6	Ordering leaks relative ranking (see Section 02).
7	Option A cannot produce the per-voter report — it has no identities.
8	Sample/seed projects are marked <code>sample: true</code> ; strip them.

BOTTOM LINE

Build Option B's experience against a real backend. The UX, copy, rules and edge cases in that file have all been reviewed and signed off — treat deviations as regressions. The three open decisions in Section 02 are the only things still genuinely undecided.

Brand Reference

Keep the built product visually identical to the prototype.

TOKEN	VALUE	USE
 Meska Blue	#2563EB	Accents, links, numbers, primary buttons
 Deep Navy	#0B1E3A	Headers, bands, dark backgrounds
 Royal Blue	#1D4ED8	Gradient ends, hot cards
 Light Blue	#93C5FD	Kickers, secondary text on dark
 Ice Blue	#EFF6FF / #DBEAFE	Card backgrounds, callouts
 Gold	#FBBF24 / #FDE68A	Winners only
 Body text	#1F2937	Paragraph text
 Muted	#6B7280 / #9CA3AF	Captions, metadata

Signature gradient: `linear-gradient(150deg, #0B1E3A 0%, #14315E 45%, #1D4ED8 130%)`

Font: Poppins — 700 titles, 500 emphasis, 300 body.

Logo: use the supplied transparent PNGs; never recreate or recolour. White wordmark on dark, dark wordmark on light.

Design details worth preserving: gradient section bands with a faint grid overlay and radial glow; rounded cards with soft shadows; pill-shaped tags; uppercase letter-spaced kickers; tabular-lining numerals for all counts.

Contacts & source material

ITEM	DETAIL
Product / requirements	Meska AI — Learning & Delivery
Source material	CEO review recording, 29 July 2026, and the original <i>Final Project Leaderboard Plan</i> PDF
Target domain	<code>leaderboard.meska.ai</code>